LIGHTCANVAS

Architectural & Technical Blueprint

A comprehensive guide to holiday lighting architecture, protocols, sequence execution, and physical integration designed to enable a modern, AI-first alternative to legacy sequencing platforms.

Document Type: Technical Strategy & Product Architecture Spec

Target Concept: AI-Native Lighting Engine

Legacy Baselines: xLights, Light-O-Rama, Falcon Player (FPP)

Date: May 10, 2026

Executive Summary & Core Mission

The modern holiday lighting hobby is restricted by a highly fragmented, technically hostile software landscape. Designing a dynamic, music-synchronized lighting show currently requires navigating legacy tools like **xLights** and **Light-O-Rama (LOR)**, configuring complex serial and network-based hardware, understanding physical protocols, and deploying custom embedded players like **Falcon Player (FPP)**.

The learning curve is steep, demanding that a homeowner act as a network engineer, a 3D CAD designer, and an audio-visual technician. For a hobbyist, getting a show up and running often represents weeks of frustrating manual configuration.

> ### The LightCanvas Vision
>
> The goal of **LightCanvas** is to build an AI-native, AI-first platform that completely abstracts this complexity. By running alongside or compiling directly into established show player ecosystems, LightCanvas replaces the manual, timeline-based sequencing grids with conversational logic, automatic computer-vision house mapping, and streamlined network synchronization.

Blueprint Outline

To implement an elegant, automated software solution, we must systematically dissect the entire ecosystem. This blueprint is divided into the following key domains:

- **Section 1: The Holiday Lighting Paradigm** — A non-technical deep dive into how computers communicate with physical light bulbs, and the standard architecture of a synchronized show.
- **Section 2: The Physical Stack (Hardware & Protocols)** — Demystifying Smart Pixels, controllers, network routing protocols (E1.31, DDP), and the role of the show player (FPP).
- **Section 3: The Data & Logic Engine** — An explanation of channels, universes, rendering mathematics, and the binary anatomy of the standard playback file (***.fseq***).
- **Section 4: The "Usual Suspects"** — The hidden pitfalls and operational pain points that frustrate users and must be programmatically automated out of existence.
- **Section 5: AI-First Technical Architecture** — The structural pillars of LightCanvas, detailing exactly how to leverage AI to automate layout, choreography, and hardware mapping.

Section 1: The Holiday Lighting Paradigm

To a regular observer, a synchronized light show looks like magic: a house outline, trees, and arches flash in perfect rhythm with music broadcast over an FM radio frequency. Under the hood, this is a highly coordinated data-streaming operation.

At its core, a holiday light show is a simple physical pipeline. It requires three distinct stages:

Phase	Legacy Process (xLights / LOR)	The AI-Native Solution (LightCanvas)
1. Design (Layout)	Draw 2D models over a photo of the house, manually calculating pixel counts and wiring directions.	Upload a photo or walk the yard with a mobile camera; computer vision auto-generates 3D models and pixel paths.
2. Sequence (Choreography)	Manually drag-and-drop individual mathematical effects onto timelines for hundreds of different props.	Describe the vibe in plain English or let the AI analyze the audio track to generate perfect rhythmic matching.
3. Playback (Show Execution)	Render raw channel files, upload to a dedicated Raspberry Pi running FPP, and map network configurations.	One-click cloud sync directly to the local controller/player via modern network discovery.

The Crucial Split: Authoring vs. Playback

The most important concept to grasp is that **design software is not playback software**.

Programs like xLights do not directly control the lights while a neighborhood show is running. Running a 3-hour light show directly from a desktop PC or laptop is highly discouraged because a standard computer is prone to operating system background tasks, Windows updates, and CPU spikes. If a PC lags by even 100 milliseconds, the lights desynchronize from the music, ruining the illusion.

Instead, the workflow is split:

1. **The Sequencer (xLights/LOR)** is the "recording studio." It is used once to design the visual layout, choose the music, place the effects, and "compile" the final animation.
2. **The Show Player (Falcon Player / FPP)** is the "MP3 player." It is a highly optimized, single-purpose micro-computer (such as a Raspberry Pi) that sits outside in a waterproof box. It takes the compiled animation file and streams raw commands to the lighting hardware with microsecond precision.

> **Product Rule #1 for LightCanvas**

> Your AI tool does not need to replace the physical show players. FPP is open-source, ultra-stable, and universally adopted. LightCanvas must focus on being the ultimate, effortless **Design & Export Engine**, generating industry-standard files that instantly load into FPP, allowing users to bypass xLights entirely.

Section 2: The Physical Stack (Hardware & Protocols)

Understanding how physical signals move from software to the individual LED bulbs is essential. To design a program that configures these systems automatically, we must dissect the hardware layers.

1. Dumb Lights vs. Intelligent Pixels

The holiday lighting world consists of two types of lights:

- **Dumb AC/DC Lights (Legacy):** These are traditional single-color light strings (like incandescent or basic LED strings plugged into a wall outlet). The entire string behaves as a single unit. It can turn on, turn off, or fade, but every bulb on the string must do the same thing at the exact same time. These are controlled using AC controllers (like Light-O-Rama's classic 16-channel metal boxes), which act as computer-controlled dimmers.
- **Intelligent Pixels (Modern Standard):** Usually called *pixels* (most commonly using the WS2811 integrated circuit chip). Every single bulb on a pixel string has its own tiny microchip inside. This allows the computer to tell Bulb #1 to turn red, Bulb #2 to turn blue, and Bulb #3 to turn green, all on a single three-wire cable. This is what enables complex patterns, video boards, and detailed text displays.

2. Controllers: The Translators

A light controller is the physical bridge. It has an Ethernet port on one side and multiple direct terminal outputs (ports) on the other.

Controllers take high-speed network protocols sent from the show player over standard Ethernet cable (Cat5/Cat6) or Wi-Fi, and convert them into the low-level serial protocol (such as the WS2811 protocol) that physical pixel strings understand.

Common controller ecosystems include:

- **Falcon Controllers (e.g., F16v4, F48):** The high-end enterprise standard. Highly reliable, supporting thousands of pixels across 16 to 48 local or remote differential ports. They possess robust web interfaces and complete HTTP APIs.
- **Kulp Controllers:** Pocket-sized, open-source boards that plug directly onto BeagleBone microcomputers. They run Falcon Player (FPP) locally and drive pixels directly from the same physical unit.
- **WLED (ESP8266/ESP32-based):** Highly popular for smaller displays, landscape lighting, and permanent roofline installations. Inexpensive, Wi-Fi enabled, and extremely easy to set up.
- **Light-O-Rama Controllers:** Legacy metal boxes built for AC lights, running over a low-speed, daisy-chained RJ45 serial connection (RS-485).

3. The Network Protocols

To stream lighting instructions over a network, the industry relies on two primary open-source languages:

A. E1.31 (SACN - STREAMING ARCHITECTURE FOR CONTROL NETWORKS)

E1.31 is an industry standard developed for theatrical stage lighting. It wraps standard DMX-512 packets (which control stage lights) into network packets (UDP). It divides data into logical containers called **Universes**. Each universe contains exactly 512 channels.

While highly robust, E1.31 has significant network overhead because it requires constant transmission of universe headers, and it forces software to artificially segment contiguous strings of pixels into arbitrary 512-channel boundaries.

B. DDP (DISTRIBUTED DISPLAY PROTOCOL)

DDP is a modern, ultra-efficient protocol designed specifically for pixel control. Instead of dividing light strings into complex "universes," DDP simply sends a raw array of pixel color data directly to an IP address.

DDP Header + [Raw Pixel Data: (R, G, B), (R, G, B)...]

DDP packets are much smaller, require almost no processing overhead on the controller, and eliminate the confusing mental math of mapping universes. It is the preferred communication protocol for modern setups.

Section 3: The Data & Logic Engine

To build a sequencer, you must understand exactly how the software handles light show configuration and compiles visual timelines into binary files.

1. The Logic Matrix: Channels & Pixels

Every single point of control in a light show is called a **Channel**.

- A single dumb AC light plug requires **1 channel** (a value from 0 to 255 representing brightness).
- A single smart pixel requires **3 channels**: one for Red, one for Green, and one for Blue.

If a user has a small "Mega Tree" prop with 16 strings of 50 pixels each, that prop contains 800 physical pixels. To control it, the computer must manage a continuous block of 2,400 individual channels:

$$16 \text{ strings} \times 50 \text{ pixels} = 800 \text{ pixels}$$
$$800 \text{ pixels} \times 3 \text{ channels (RGB)} = 2{,}400 \text{ channels}$$

2. The Render Pipeline

When a user places an effect (like a "Rainbow" or "Butterfly") onto a prop in a design tool, the software does not save the effect as "Rainbow" in the final playback file. The software must calculate—frame by frame—what color every single pixel should be.

Standard light shows run at a frame rate (or "step time") of either **25 milliseconds (40 frames per second)** or **50 milliseconds (20 frames per second)**.

Data Explosion Example:
Consider a modest home display with 10,000 pixels (30,000 channels) running a 3-minute (180 seconds) song at 40 FPS:

- Total Frames: $180 \text{ seconds} \times 40 \text{ FPS} = 7{,}200 \text{ frames}$
- Total Output Data Points: $30{,}000 \text{ channels} \times 7{,}200 \text{ frames} = 216{,}000{,}000 \text{ bytes (216 MB)}$

Every frame, the software must calculate and stream 30,000 bytes. This raw matrix of numbers is what the show player reads.

3. File Formats: The Recipe vs. The Meal

To sit beside xLights or LOR, your software must read and write two distinct types of files:

A. THE DESIGN FILE (.XSQ IN XLIGHTS, .LMS/.LCC/LOREDIT IN LOR)

This is an **XML file**. It is human-readable and represents the "recipe" of the sequence. It describes the timing tracks, the models, and the abstract effects placed on the timeline. It does *not* contain raw lighting commands. If you double-click this file, a player cannot play it; it must be rendered first.

B. THE PLAYBACK FILE (.FSEQ)

The **Falcon Sequence (.fseq)** file is the compiled, raw, binary "meal." This file contains no information about "effects," "timelines," or "models." It is a massive binary grid of raw numbers.

A show player (FPP) opens an ***.fseq*** file, reads the header to find out how many channels exist per frame, and then starts a timer. Every 25ms, it grabs the next chunk of bytes and dumps them directly onto the network via E1.31 or DDP.

ANATOMY OF AN FSEQ FILE (V2 SPECIFICATION)

To program an export module, your developers must strictly write to the FSEQ v2 binary specification. The file is structured as follows:

Byte Offset	Data Type	Description
0 - 3	4-byte String	Magic identifier: Must be ***ext{'PSEQ'}*** (or Falcon Sequence).
4 - 5	uint16 (Little Endian)	Channel Data Offset: Where the raw light frames actually begin.
6	uint8	Minor version (typically 0).
7	uint8	Major version (typically 2).
10 - 13	uint32 (Little Endian)	Channel Count per Frame: The exact size of one frame of data.
14 - 17	uint32 (Little Endian)	Number of Frames: Total length of the sequence.
18	uint8	Step Time: Frame duration in milliseconds (e.g., 25 or 50).
20	uint8	Compression Type: 0 = None, 1 = ZStandard (zstd), 2 = zLib.

Section 4: The "Usual Suspects" (Hidden Pitfalls)

Why do hobbyists spend weeks struggling with current software? Because legacy tools expose raw hardware configurations directly to the user. To build an AI-native product, we must automate the following hidden pain points:

1. The Network Configuration Nightmare

In xLights, a user must manually align three different maps:

1. The **Visual Prop Map** (how many pixels are on a physical Arch).
2. The **Software Controller Map** (mapping that Arch to Controller #1, Port #4, start channel 4501, end channel 4800).
3. The **Physical Controller Board Settings** (manually logging into a hardware web browser portal at ***ext{192.168.1.100}*** and typing in the exact same channel settings for Port #4).

If a single digit is mismatched anywhere in this chain, the entire display breaks or lights flicker wildly.

2. Unmanaged Frame Collisions (Multicast vs. Unicast)

When streaming thousands of channels of data over a network, the choice of transmission type is critical:

- **Multicast:** The player blasts data to every device on the network. While easy to set up (no IP addresses required), it quickly saturates home Wi-Fi routers, causing lagging lights, dropped signals, and household internet crashes.
- **Unicast:** The player sends data directly to specific IP addresses. This is highly efficient, but requires the user to manage static IP addresses for every controller board.

3. The Render Cache & "Save" Delays

Because rendering millions of data points takes time, xLights utilizes a local "Render Cache."

> **The "Save" Pitfall**
>
> If a user saves a sequence in xLights without clicking "Render All" first, the actual ***.fseq*** file will not update, meaning edits made on screen will not appear when the show is played. Users frequently complain: *"I saved my changes, but the lights outside are still playing the old version!"* Your engine must treat rendering as an automated, background-compiled process that requires zero manual intervention.

Section 5: AI-First Technical Architecture (The LightCanvas Blueprint)

To replace xLights and LOR, LightCanvas must not be a direct clone with a prettier interface. It must use AI to entirely bypass the legacy manual processes. Here is how we architect the core pillars of LightCanvas:

Pillar 1: AI-Powered Computer Vision Layout

Instead of manually drawing pixel lines on a 2D canvas, the user takes a photo or a brief video scan of their house with a smartphone.

- **Under-the-Hood Logic:** An object detection and edge-estimation neural network (e.g., custom YOLO or Segment Anything Model) detects structural features: rooflines, windows, doors, columns, and walkways.
- **Automatic Prop Fitting:** The software automatically lays down a proposed pixel layout. It suggests: *"We detected a 30-foot horizontal roofline. Placing a 150-pixel string here."*
- **Calibration Walk:** The user plugs their lights into the controllers, and the software lights up pixels sequentially. By pointing the camera at the active lights, the AI instantly maps physical pixel positions to the virtual layout. No manual coordinate mapping is required.

Pillar 2: AI Multi-Modal Audio Choreography

Instead of spending 40 hours drag-dropping colors onto a grid, the user uploads an MP3.

- **The AI Listener:** The system runs an audio feature extraction model (analyzing onset strength, spectral flux, pitch, tempo, and vocal tracks).
- **Thematic Prompts:** The user inputs a text prompt: *"Make this high-energy, using neon blues and pink sweeps on the arches, and sync the singing Christmas trees to the lead vocals."*
- **Synthesis Engine:** The LLM translates the prompt and audio features into a structural sequence plan, dynamically placing transitions on beat drops and mapping vocal tracks directly to phoneme shapes on face-props (singing trees).

> #### The Magic Leap: Auto-Compilation to Binary
>
> The output of this AI pipeline is translated directly into the binary ***.fseq*** structure in the background. The user simply watches a 3D rendering of their house, gives feedback in real-time chat (e.g., *"Make the roofline flash green during the chorus"*), and the system instantly re-compiles the file.

Pillar 3: Zero-Config Auto-Discovery & DDP-First Networking

LightCanvas completely eliminates the manual IP-and-Universe nightmare.

- **mDNS/SSDP Controller Discovery:** When launched on a home network, the software scans for local devices. It automatically discovers Falcon, Kulp, and WLED controllers via network broadcasts.

- **API Integration:** Instead of making the user log into web browser portals, LightCanvas communicates directly with the controller APIs (such as WLED's JSON API or Falcon's HTTP endpoints) to pull the hardware port details and push the correct pixel configurations automatically.
- **DDP-First Streaming:** By prioritizing the Distributed Display Protocol (DDP), LightCanvas completely hides the concept of "Universes" from the user. It treats the physical display as a continuous canvas of pixels mapped to IP addresses, entirely eliminating the mathematical overhead of theatrical DMX configurations.

Summary of the LightCanvas Architecture

By combining physical-level compilation with top-level AI automation, LightCanvas establishes a highly streamlined design-to-execution pipeline:

```
[1. Visual Scan] --(AI Computer Vision)--> [2. Auto-3D Prop Layout]
                                                    |
[4. Compiled .fseq] <--(AI Choreography Engine)-- [3. Audio Feature Extraction]
        |
        +--(mDNS Auto-Discovery / DDP Protocol)--> [5. Physical Controller Outputs]
```